

34. Jazyk UML

34. Jazyk UML

SZZ okruh 34 (FIT BUT). Standardizovaný grafický jazyk pro vizualizaci a návrh systémů.

Shrnutí

Diagramy struktury

- Popisují **statické** aspekty (z čeho je systém složen).
- **Diagram tříd** — třídy (jméno, atributy, operace + viditelnost public/private/protected) a vztahy:
 - **Asociace** (základní vztah), **agregace** (volná vazba celek–část, prázdný kosočtverec — část přežije celek), **kompozice** (silná, plný kosočtverec — část bez celku nezanikne), **generalizace/dědičnost**, **realizace** (třída $\square$ rozhraní), **závislost** (stereotypy «use», «create»...).
 - **Mohutnost (multiplicity)** = kolik instancí jedné třídy se váže k instanci druhé.
- Dále: diagram objektů, komponent, seskupení (balíčky).

Diagramy chování

- Popisují **dynamické** aspekty (jak systém funguje).
- **Use case** (aktéři, případy užití; «**include**» povinný, «**extend**» volitelný; generalizace).
- **Diagram aktivit** (vývojový diagram — větvení, paralelní toky), **stavový diagram** (životní cyklus objektu ~ [[okruhy/03-sekvencni-logicke-obvody|stavový automat]]).

Diagramy interakce

- Podskupina chování. **Sekvenční diagram** (čáry života, časově řazené zprávy; plná šipka = volání, čárkovaná = odpověď, nevyplněná = asynchronní), **diagram komunikace** (struktura, hierarchické číslování zpráv).

UML v procesu vývoje viz [[okruhy/33-zivotni-cyklus-sw|RUP]]; ER × diagram tříd viz [[okruhy/35-konceptualni-modelovani-db]]; OO koncepty viz [[okruhy/40-objektova-orientace]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Obecné $\square$ [[#Diagramy struktury]] - *Co je UML, k čemu?* $\square$ Standardizovaný grafický jazyk pro vizualizaci, specifikaci a dokumentaci systémů (podpora OO analýzy). - *Typy diagramů?* $\square$ Strukturní (statika) × behaviorální (dynamika; interakční jako podskupina).

Diagram tříd $\square$ [[#Diagramy struktury]] - *Asociace / agregace / kompozice / generalizace?* $\square$ Asociace = vztah; agregace = volný celek–část (část přežije); kompozice = silná (část zaniká s celkem); generalizace = dědičnost. - *Viditelnost?* $\square$ public (kdokoli), private (jen třída), protected (třída + potomci).

Use case □ [[#Diagramy chování]] - *Include* vs. *extend*? □ «include» povinné (vždy se provede), «extend» volitelné/podmíněné rozšíření.

Sekvenční diagram □ [[#Diagramy interakce]] - *Co zobrazuje?* □ Časově řazené zprávy mezi objekty (čáry života); synchronní (plná) × asynchronní (nevyplněná šipka).

Plné znění (ke studiu)

Unified Modeling Language je v softwarovém inženýrství **standardizovaný grafický jazyk** pro vizualizaci, specifikaci, navrhování a dokumentaci programových systémů. UML nabízí standardní způsob zápisu a podporuje **objektově orientovaný přístup k analýze**. Základním cílem je **vizualizace, specifikace struktury a chování navrhovaného systému**. Napomáhá k dekompozici systému. Umožňuje modelovat **business procesy**, konkrétní **příkazy programovacího jazyka**, vztahy mezi třídami v OOP. UML diagramy dělíme do dvou skupin (respektive do 3, **ERD do UML NEPATŘÍ**):

- **Structure diagrams** (Diagramy struktury)
- **Behaviour Diagram** (Diagramy chování)
- **Interaction diagrams** (Diagramy interakce) - podskupinou diagramů chování

Diagramy struktury (Structure diagrams)

Popisují strukturu systému, tedy **z čeho je systém složený**. Představují **statické aspekty** systému. Zdůrazní části/komponenty, které musí být přítomny v modelovaném systému. Jsou široce používány při **dokumentaci architektury softwarových systémů**. **Diagram komponent** například popisuje, jak je softwarový systém rozdělen na **komponenty**, a ukazuje **závislosti** mezi těmito komponentami.

Diagram tříd

Zobrazuje **třídy** a **statické vztahy** mezi nimi. Popisuje **statickou strukturu systému** a znázorňuje datové struktury. Každá třída je tvořena **jménem**, **seznamem atributů** a **seznamem operací**. Atributy a operace (metody, funkce) mají stanovenou **viditelnost**. Atributy mají navíc specifikovaný **typ** a operace mají navíc **vstupní** [[media/szz-34/media/image29.png]] **parametry** a **výstupní typy**. Diagramem tříd se popisuje **doménový model**.

Mohutnost (Multiplicity) Mohutnost vztahu udává, kolik instancí jedné třídy může být svázáno s instancí třídy druhé.

Viditelnost [[media/szz-34/media/image3.png]]

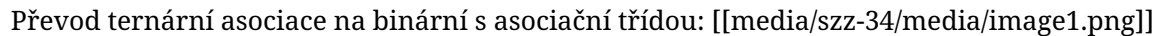
Viditelnost atributů a operací udává, které atributy a operace lze používat mimo třídy, buď v odvozených třídách nebo i mimo třídu a její podtřídy.

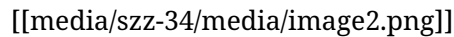
- **veřejný (public)**: dostupné komukoliv
- **soukromý (private)**: dostupné jen v rámci dané třídy
- **chráněný (protected)**: dostupné v rámci dané třídy a jejích potomků, pro zbytek světa se chová jako soukromé

[[media/szz-34/media/image17.png]]

[[media/szz-34/media/image20.png]] **Asociace**

Asociace určuje **základní vztah** mezi dvěma entitami. Ty mohou existovat nezávisle na sobě. Zakresluje ji jednoduchou plnou čáru. Mohou mít název (sloveso, podstatné jméno). Asociace mohou být **binární** (unární je také brána jako binární ale **reflexivní**), **ternární** a **n-ární**. Jiné než binární asociace není moc doporučeno používat (nepřehledné, obvykle lze převést na binární nebo na asociační třídu).

Převod ternární asociace na binární s asociační třídou: 



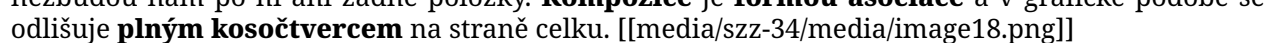


Dědičnost (Generalization) Vyjadřuje vztah generalizace/specializace mezi třídami. Odvozená třída **sdílí atributy** (dle viditelnosti) **chování** (operace), **vztahy** a **omezení** obecnější třídy. Odvozená třída může navíc přidávat a **modifikovat** atributy a chování (operace).

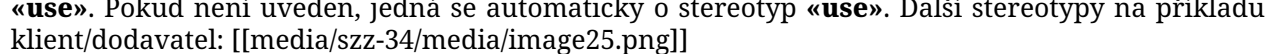
Agregace (Aggregation) 

Agregace představuje **volnou vazbu mezi celkem a součástí**, kdy jeden objekt (**celek**) využívá služby dalších objektů (**součástí**). Například vztah mezi počítačem a tiskárnou je vztah typu **agregace**, kdy počítač s tiskárnou tvoří jeden celek, ale **tiskárna může existovat i bez počítače**. Dokonce může být **součást** (tiskárna) využita i jiným **celkem** (např. mobil) - může být součástí více kolekcí. Agregace je **formou asociace** a v grafické podobě se odlišuje **prázdným kosočtvercem** na straně celku.

Kompozice (Composition) 

Kompozice je podobná agregaci, avšak reprezentuje **silnější vztah**. Entita části (součást) **nemá bez celku smysl**, nemůže bez celku existovat. Pokud zanikne celek, zanikají automaticky i **jeho části**. Příkladem kompozice je vztah mezi fakturou (**celek**) a jejími položkami (**součásti**). Každá položka **musí být součástí právě jedné faktury**, faktura má jednu a více položek. Jestliže fakturu zrušíme, nezbudou nám po ní ani žádné položky. **Kompozice** je **formou asociace** a v grafické podobě se odlišuje **plným kosočtvercem** na straně celku. 

Závislost (Dependency) Závislost je slabší forma vztahu, která naznačuje, že **jedna třída závisí na jiné**.

Závislost bývá doplněna o **stereotyp**, který ji blíže specifikuje. Nejčastěji se používá stereotyp «**use**». Pokud není uveden, jedná se automaticky o stereotyp «**use**». Další stereotypy na příkladu klient/dodavatel: 



- «**initiate**» / «**create**»: klient vytváří instance dodavatele,
- «**send**»: operace klienta zasílá signál příjemci (dodavateli),
- «**call**»: klientská třída volá operaci dodavatele,
- «**trace**»: klient realizuje dodavatele,
- «**refine**»: klientská třída poskytuje detailnější informace než dodavatel.

Realizace Udává vztah mezi **třídou** a **rozhraním** (interface). Označuje fakt, že třída (zde *Zákazník/Objednávka*) **implementuje všechny operace z daného rozhraní** (respektive implementuje rozhraní) (zde *Tisk*). Objekt používající rozhraní (zde *Tiskárna*) pak umí používat i tuto (implementační) třídu.

Diagram objektů (Object Diagram)

[[media/szz-34/media/image23.png]]

Je úzce svázán s diagramem tříd, **znázorňuje objekty a jejich relace** (vztahy) **v určitém čase**. Relace jsou **dynamické** (nemusí trvat po celou dobu existence objektu). [[media/szz-34/media/image22.png]]

Při vytváření objektů dochází k vytváření **spojení** mezi nimi. Spojení jsou **instance asociací**. [[media/szz-34/media/image11.png]]

Diagram komponent

Diagram komponent znázorňuje **komponenty** použité v systému. Komponentami mohou být **logické komponenty** (např. business komponenty, procesní komponenty) nebo také **fyzické komponenty**. Popisuje, jak je softwarový systém rozdělen na komponenty a ukazuje **závislosti mezi těmito komponentami**. [[media/szz-34/media/image10.png]]

Diagram seskupení

Umožňuje **seskupit sémanticky související elementy**, umožňuje **zapouzdřit prostor jmen**. Definiuje sémantické hranice modelu a umožňuje souběžnou práci v etapě návrhu. Např. třídy a rozhraní řešící komunikaci dáme do jednoho balíčku a třídy obsluhující klienta do druhého.

Diagramy chování (Behaviour diagrams)

[[media/szz-34/media/image16.png]]

Popisují **chování systému**, tedy **jak systém funguje**. Diagramy chování představují **dynamický aspekt** systému. Zdůrazňují, co se může stát v modelovaném systému. Jsou široce používány k popisu **funkčnosti softwarových systémů**. Diagram případu užití například zobrazuje uživatele v nějaké roli a operace, které může provádět.

Diagram případů užití

Diagram případů užití zachycuje vnější pohled na modelovaný systém, specifikujeme pomocí něj **účastníky** a způsoby, jak budou modelovaný **systém používat** - případy užití. Používá se v Use-Case driven přístupech (metodika RUP; Rational Unified Process). Prvky diagramů případů užití:

- **hranice systému**,
- **účastník (aktér/actor)**: subjekt (většinou uživatel, může mít i speciální podobu, např. čas, jiný systém), který se systémem pracuje,
- **případ užití**: funkce, kterou systém vykonává jménem (akcí) jednotlivých účastníků nebo v jejich prospěch,
- **interakce**: ukazuje účast autora (účastníka) na provádění případu užití.

Asociace mezi účastníkem a případem užití Účastník musí být pomocí asociace spojen aspoň s jedním případem užití. Více účastníků může být asociováno s jedním případem užití (viz obrázek na konci kapitoly).

Generalizace účastníka (dědičnost) Zobecnění účastníka znamená, že jeden účastník může **zdědit roli druhého**. Potomek **zdědí všechny případy použití předka**. Potomek má **navíc** jeden nebo více případů použití, které jsou pro něj specifické (pro danou roli), a předek ji nemá.

Generalizace případu užití [[media/szz-34/media/image5.png]]

Je to podobné jako zobecnění účastníka. Chování předka dědí potomek. Používá se, když existuje **společné chování mezi dvěma případy užití** a také **specializované chování** specifické pro každý případ použití. Viz příklad na obrázku. [[media/szz-34/media/image21.png]]

Extend mezi dvěma případy užití Vazba extend **rozšiřuje funkcionalitu** daného případu užití (přidává do systému další funkcionalitu). Případ užití vázaný pomocí extend (rozšiřující případ užití) se **může provést, ale nemusí** (záleží na okolnostech, případně se může provádět pouze **podmíněně**). Rozšiřující případ užití je **závislý** na rozšiřovaném případě užití (sám o sobě nedává smysl). Naopak **rozšiřovaný případ užití musí dávat** sám o sobě **smysl**. [[media/szz-34/media/image27.png]]

Include mezi dvěma případy užití Zahrnutý/přiložený (included) případ použití **je povinný a není volitelný**. Základní případ použití je **bez přiloženého případu použití neúplný**. Používá se v případě, že je nějaká funkcionalita **důležitá natolik**, že nemůže být součástí nějakého případu užití. Chceme jí tímto zdůraznit. [[media/szz-34/media/image26.png]]

Detaily případů užití Slouží pro konkretizaci use case. Často se používá **tabulka** - má vstupní podmínky a tok událostí. [[media/szz-34/media/image9.png]]

Příklad diagramu případů užití [[media/szz-34/media/image6.png]]

Diagram aktivit (Activity diagram)

Reprezentuje objektově orientovaný **vývojový diagram**. Umožňuje modelování:

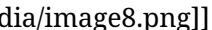
- scénářů případů užití,
- detailů **algoritmů** a operací (funkcí),
- modelování **obchodních procesů**.

Prvky diagramu:

- **uzly**:
 - **akční uzly**: modelují aktivitu,
 - **řídící uzly**: modelují rozhodování, dále např. počáteční uzel, koncový uzel,
 - **objektové uzly**: modelují objekty podílející se na aktivitách.
- **hrany**:
 - **řídící hrany**: modelují přechody mezi uzly,
 - **objektové hrany**: modelují cesty objektů mezi uzly.

Možnosti modelování

- tok událostí a dat,
- rozhodování,
- větvení a slučování,

- iterace,
- paralelní toky. 

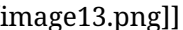


Stavový diagram

Umožňují modelování životního cyklu jednoho reaktivního (měnící se) objektu. Je to zvláštní případ stavového automatu. Mohou také modelovat dynamické chování těchto objektů:

- třídy, respektive instance tříd (objekty),
- případy užití,
- podsystémy,
- systémy.

Stavový diagram tvoří:

- **stavy**,
- **přechody**, 



- **události**: udávají, kdy dojde k přechodu z jednoho stavu do druhého. Události mohou být i ve stavech.

Reaktivní objekt

- reaguje na vnější události,
- životní cyklus je modelován jako řada stavů, přechodů a událostí,
- chování je důsledkem předchozího chování, následující stav závisí na předchozím stavu.

Diagramy interakce (Interaction diagrams)

Jedná se o **podskupinu** diagramů chování. Popisují **interakci mezi jednotlivými částmi systému** (včetně uživatele). Zdůrazňují **tok řízení** a dat mezi částmi v modelovaném systému. Sekvenční diagram například ukazuje, jak spolu objekty komunikují pomocí **sekvence zpráv**.

Sekvenční diagram (Sequence diagram)

Zdůrazňuje časově orientovanou posloupnost předávání zpráv mezi objekty (chronologické zaslání zpráv). Sekvenční diagram nejčastěji znázorňuje spolupráci několika vzorových objektů v rámci jednoho případu užití. Sekvenční diagramy jsou většinou přehlednější a srozumitelnější než diagramy komunikace. Sekvenční diagram tvoří:

- **čáry života** (lifelines): zobrazuje časovou osu určitého objektu, běh času,
- **vodorovné šipky**: reprezentují zprávy posílané mezi objekty, existuje více typů: **Plné šipky** značí volání, přerušované pak odpověď. **Nevyplněná šipka** značí asynchronní zprávu -> odesílatel nečeká na odpověď a pokračuje v činnosti.

Jednotlivé procesy či objekty zapojené do popisovaného případu užití jsou umístěny v horní části diagramu. Podlouhlé obdélníky na svislých čarách vyznačují dobu zpracovávání dané zprávy či čekání na odpověď.

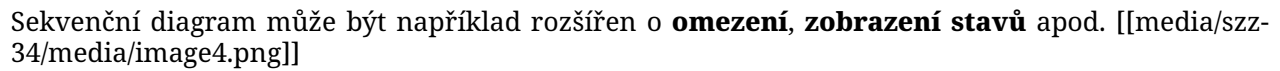
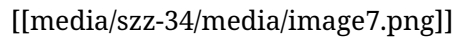
Sekvenční diagram může být například rozšířen o **omezení, zobrazení stavů** apod. 

Diagram komunikace



Zdůrazňují strukturální vztahy mezi objekty, jsou vhodné spíše pro rychlé zobrazení komunikace mezi objekty. Jsou méně přehledné než sekvenční diagramy.

- objekty jsou **spojeny linkami**,
- zprávy jsou řazeny podle **hierarchického číslování**.



Diagramy komunikace mohou být doplněny o **větvení, kontrolní podmínky** atd.



Základní pohledy

Projekce systému na jeden z jeho klíčových aspektů.

- **Strukturální** - Popisuje vrstvu mezi objekty a třídami, jejich asociace a možné komunikační kanály.
- **Datový** - Popisuje stavy systémových komponent a jejich vazby.
- **Pohled na chování** - Popisuje, jak systémové komponenty interagují a charakterizuje reakce na vnější systémové operace.
- **Pohled na rozhraní** - Je zaměřeno na zapouzdření systémových částí a jejich potenciální použití okolím systému.

Zdroje

- SZZ okruh 34 — studijní materiály FIT BUT (szz-34.docx). Obrázky: media/szz-34/.

Navigace: [\[\[index| • Přehled okruhů\]\]](#) · [\[\[okruhy/33-zivotni-cyklus-sw|33. Životní cyklus softwaru\]\]](#) · další: [\[\[okruhy/35-konceptualni-modelovani-db|35. Konceptuální modelování a návrh DB\]\]](#)
[\[\[](#)